МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет по курсовой работе по дисциплине "Функциональное программирование" Реализация проекта на языке Haskell по теме "Визуализатор маршрутов"

Отчет выполнил
студент группы 5130201/40001 _____ Костина А.В.
Отчет принял
преподаватель _____ Моторин Д.Е.

«____»_____ 2026г.

Санкт-Петербург, 2026

Содержание

Техническое задание **4**

Введение **12**

1 Особенности реализации **13**
1.1 Описание разработки проекта 13
1.1.1 Структура проекта 13
1.1.2 Диаграмма модулей 13
1.2 Описание технических особенностей реализации проекта 15
1.2.1 Типы данных 15
1.2.2 Алгоритм Дейкстры в чисто функциональном стиле 15
1.2.3 Вычисление расстояний по координатам 15
1.2.4 Парсинг и генерация JSON с помощью библиотеки aeson . 16
1.2.5 Логирование с ротацией файлов 16
1.2.6 Визуализация через Graphviz 16
1.2.7 Консольный интерфейс и обработка ошибок 17
1.2.8 Тестирование с использованием QuickCheck 17

2 Примеры работы программы **18**
2.1 Главное меню 18
2.2 Загрузка графа из JSON-файла 18
2.3 Просмотр списка станций 19
2.4 Поиск станции по части названия 20
2.5 Поиск пути с пересадкой 20
2.6 Поиск кратчайшего пути 20
2.7 Визуализация графа 22
2.8 Визуализация пути 22
2.9 Просмотр и очистка лога 22
2.10 Экспорт графа в JSON 23

3 Руководство по эксплуатации для пользователя **24**
3.1 Входные данные 24
3.2 Запуск программы 25
3.3 Загрузка графа из JSON-файла 25
3.4 Экспорт графа в JSON-файл 26
3.5 Просмотр списка станций 26
3.6 Поиск станции по части названия 26
3.7 Поиск кратчайшего пути 26
3.8 Визуализация графа 26
3.9 Визуализация пути 26
3.10 Просмотр лога 27

3.11 Очистка лога 27
3.12 Выход из программы 27
3.13 Обработка ошибок 27

Заключение **28**

Список литературы **30**

Техническое задание

1. Основание для выполнения работы

Работа выполняется в соответствии с учебным планом по дисциплине «Функциональное программирование» в рамках курсового проектирования студента группы 5130201/40001 Института компьютерных наук и кибербезопасности Федерального государственного автономного образовательного учреждения высшего образования «Санкт-Петербургский политехнический университет Петра Великого», обучающегося по направлению подготовки 02.03.01 Математика и компьютерные науки.

2. Исполнитель и соисполнители работы

Исполнитель: студент группы 5130201/40001
Соисполнители: отсутствуют

3. Цели, задачи и исходные данные

a. Цель работы

Целью работы является разработка наглядного инструмента для поиска и визуализации кратчайших маршрутов в транспортных сетях (метрополитен, железные дороги).

b. Задачи работы

В процессе выполнения работы требуется решить следующие задачи:

- **Чтение и запись файлов:**
  - реализовать импорт графа из JSON-файла, описывающего схему метро/ЖД с использованием библиотеки `aeson`;
  - реализовать экспорт графа в JSON-файл;
  - реализовать сохранение визуализаций графа и найденного кратчайшего пути в PNG-файлы.
- **Обработка информации математическими функциями:**
  - разработать структуры данных для представления взвешенных графов (ориентированных/неориентированных);

  - реализовать алгоритм Дейкстры для поиска кратчайшего пути;
  - реализовать восстановление маршрута от начальной до конечной вершины.

- **Взаимодействие с пользователем через консоль:**
  - разработать систему меню для управления программой;
  - обеспечить ввод начальной и конечной станций для построения маршрута.

- **Логирование действий пользователя:**
  - реализовать запись в лог-файл всех значимых действий;
  - добавить временные метки и уровни логирования.

- **Обработка ошибок:**
  - обеспечить корректную обработку ошибок ввода/вывода;
  - обрабатывать некорректный пользовательский ввод;
  - обрабатывать ситуации, когда путь между станциями отсутствует.

c. Исходные данные

Исходными данными для проведения работы являются:

- JSON-файлы с описанием графов транспортных сетей в формате, включающем названия станций и перегоны с весами;
- готовые датасеты транспортных сетей (например, из открытых источников);

4. Основное содержание работы

a. Разработка проекта

Разработка проекта включает:

- проектирование архитектуры и системы типов;
- реализацию модулей для работы с графами, алгоритмом Дейкстры, визуализацией, вводом/выводом JSON, логированием и обработкой ошибок;
- написание тестов с использованием QuickCheck;
- отладку и тестирование программы.

b. Отчетная документация

Отчёт и программа на языке Haskell.

5. Основные требования к выполнению работы

a. Назначение разрабатываемого программного комплекса

Программный комплекс предназначен для поиска и визуализации кратчайших маршрутов в транспортных сетях (метро, ЖД). В зависимости от выбора пользователя оценивает время прохождения или длину маршрута по транспортным сетям. Может использоваться как вспомогательный инструмент для планирования поездок.

b. Состав разрабатываемого программного комплекса

- Система сборки: Stack;
- JSON-файлы с примерами схем метро и ЖД;
- документация пользователя.

c. Технические характеристики

Файлы, используемые программой:

- JSON-файл с описанием транспортной сети (станция, линия, координаты); Например:

```
{
 "lines": [
 {
  "hex_color": "CD0505",
  "stations": [
  {
   "lat": 53.20969,
   "lng": 50.13442,
   "order": 0,
   "name": "Алабинская",
```

```
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2113838,
 "lng": 50.15022,
 "order": 1,
 "name": "Российская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2037926,
 "lng": 50.159832,
 "order": 2,
 "name": "Московская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2004433,
 "lng": 50.1765938,
 "order": 3,
 "name": "Гагаринская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2011223,
 "lng": 50.1992874,
 "order": 4,
 "name": "Спортивная",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2017365,
 "lng": 50.2206573,
```

```
 "order": 5,
 "name": "Советская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2073135,
 "lng": 50.2364159,
 "order": 6,
 "name": "Победа",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2129974,
 "lng": 50.24889,
 "order": 7,
 "name": "Безымянка",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2113533,
 "lng": 50.2697754,
 "order": 8,
 "name": "Кировская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2127,
 "lng": 50.28298,
 "order": 9,
 "name": "Юнгородок",
 "admArea": null,
 "district": null,
 "status": null
}
],
```

```
    "id": "54",
    "name": "Первая"
   }
   ],
   "name": "Самара"
  }
```

Исходные данные получены из репозитория: `https://github.com/morphey83/MetroAndDistrict/blob/master/metro.all.json` (Дата обращения 03.04.2026)

- JSON-файл для экспорта текущей схемы;
- PNG-файл визуализации всей схемы;
- PNG-файл визуализации найденного маршрута;
- текстовый файл лога с временными метками и уровнями логирования.

Что технически может сделать пользователь:

- Загрузить схему из JSON-файла;
- Экспортировать текущую схему в JSON-файл;
- Просмотреть список всех станций;
- Найти станцию по части названия;
- Выбрать начальную и конечную станцию;
- Запустить алгоритм Дейкстры для поиска кратчайшего пути;
- Просмотреть маршрут и его длину;
- Сохранить визуализацию всей схемы в PNG;
- Сохранить визуализацию маршрута с выделенным путём в PNG;
- Просмотреть последние записи лога или весь лог;
- Очистить лог-файл.

d. Функциональные требования

d.1. Требования к производительности

- Загрузка и парсинг JSON-файла — не более 2 секунд.
- Работа алгоритма Дейкстры для графа до 500 вершин — не более 0.5 секунды.

- Генерация PNG-визуализации — не более 3 секунд.
- Потребление оперативной памяти — не более 200 МБ.

d.2. Требования к точности и обработке данных

- Алгоритм Дейкстры должен находить кратчайший путь с точностью 100%.
- Веса рёбер поддерживаются дробными (точность до 0.1).
- При импорте JSON выполняется проверка ссылочной целостности.
- Программа корректно обрабатывает неориентированные графы.

d.3. Требования к хранению и логированию

- Все действия пользователя фиксируются в лог-файле.
- Максимальный размер лога — 10 МБ, при превышении старая запись перезаписывается.
- Экспорт сохраняет полную структуру графа без потерь.

d.4. Требования к надёжности

- Программа не должна завершаться аварийно при некорректном вводе.
- При отсутствии пути между станциями выводится сообщение об ошибке.
- Визуализация невозможна без загруженной схемы.

e. Стандартные средства

При разработке должны использоваться типовые средства языка Haskell и существующие библиотечные решения:

- `aeson` — для парсинга и генерации JSON-файлов;
- `containers` (Data.Map, Data.Set, Data.Sequence) — для представления графов и структур данных алгоритмов;
- `graphite` — для визуализации графов и сохранения в PNG;
- `QuickCheck` — для тестирования свойств;
- `directory` — для работы с файловой системой;
- `time` — для формирования временных меток в логах;
- `bytestring` — для эффективной работы с JSON.

6. Этапы и сроки выполнения работы

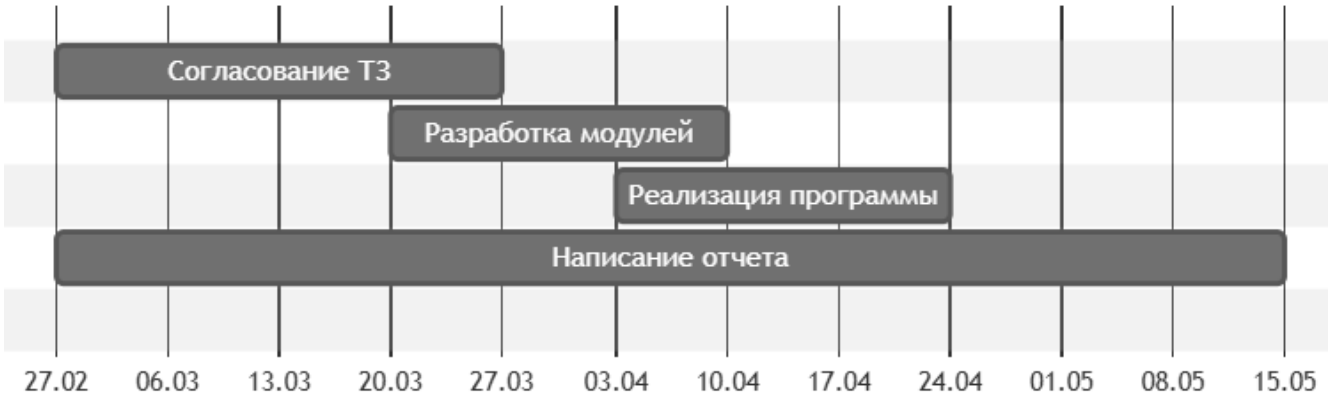


Рис. 1: Диаграмма Ганта этапов работ

7. Результаты работы

1. Утверждённое техническое задание.
2. Диаграмма модулей, описание типов данных, обоснование выбора библиотек.
3. Работающие модули парсинга JSON, поиска маршрута (алгоритм Дейкстры), визуализации, консольного интерфейса, логирования и обработки ошибок.
4. Готовый отчёт с описанием разработки, примерами работы и руководством пользователя.

Введение

Современные транспортные системы крупных городов представляют собой сложные инженерные и инфраструктурные объекты. Метрополитен как внеуличный пассажирский транспорт играет ключевую роль в обеспечении мобильности населения мегаполисов [1]. Эффективное планирование поездок в таких сетях требует автоматизированных инструментов, способных быстро находить оптимальные маршруты между заданными станциями.

Математическое моделирование транспортных процессов позволяет формализовать задачу поиска маршрутов и найти её эффективное решение [4]. Согласно исследованиям в области транспортной логистики, маршрутизация пассажирских перевозок требует учёта множества факторов: времени в пути, количества пересадок, загруженности линий [2]. Для формализации транспортной сети наиболее естественным является применение теории графов, где станции выступают вершинами, а соединяющие их перегоны — рёбрами [3].

Вопросы оценки роли метрополитена в транспортных системах крупных городов рассматриваются в работах Н.В. Булычевой, Д.В. Капского и других исследователей, которые применяют методы математического моделирования для анализа эффективности работы подземных транспортных систем [5]. Особый интерес представляет геоинформационное моделирование транспортной доступности, позволяющее оценить интегральные характеристики маршрутных сетей [6].

Классическим алгоритмом решения задачи о кратчайшем пути во взвешенных графах является алгоритм, предложенный Эдсгером Дейкстрой. Этот алгоритм широко применяется в системах навигации и маршрутизации благодаря своей эффективности и относительной простоте реализации.

Таким образом, сочетание методов теории графов, математического моделирования транспортных процессов и современных средств разработки программного обеспечения позволяет создать инструмент для визуализации и поиска оптимальных маршрутов в транспортных сетях, что и является целью данной работы.

1 Особенности реализации

1.1 Описание разработки проекта

1.1.1 Структура проекта

```
route-visualizer/
  app/
    Main.hs               # Точка входа
  src/
    CLI/
      Menu.hs             # Консольный интерфейс
    Graph/
      Types.hs            # Типы данных
      Loader.hs           # Загрузка/экспорт JSON
      Dijkstra.hs         # Алгоритм Дейкстры
    Visualization/
      Render.hs           # Визуализация в PNG
    Logging/
      Logger.hs           # Логирование
  test/
    Spec.hs               # Тесты QuickCheck
  data/
    metro.json            # Входные данные
  logs/
    app.log               # Лог-файл
  output/
    full_graph.png
    route.png
  package.yaml            # Stack конфигурация
  stack.yaml
```

1.1.2 Диаграмма модулей

На Рис.2 изображена диаграмма модулей.

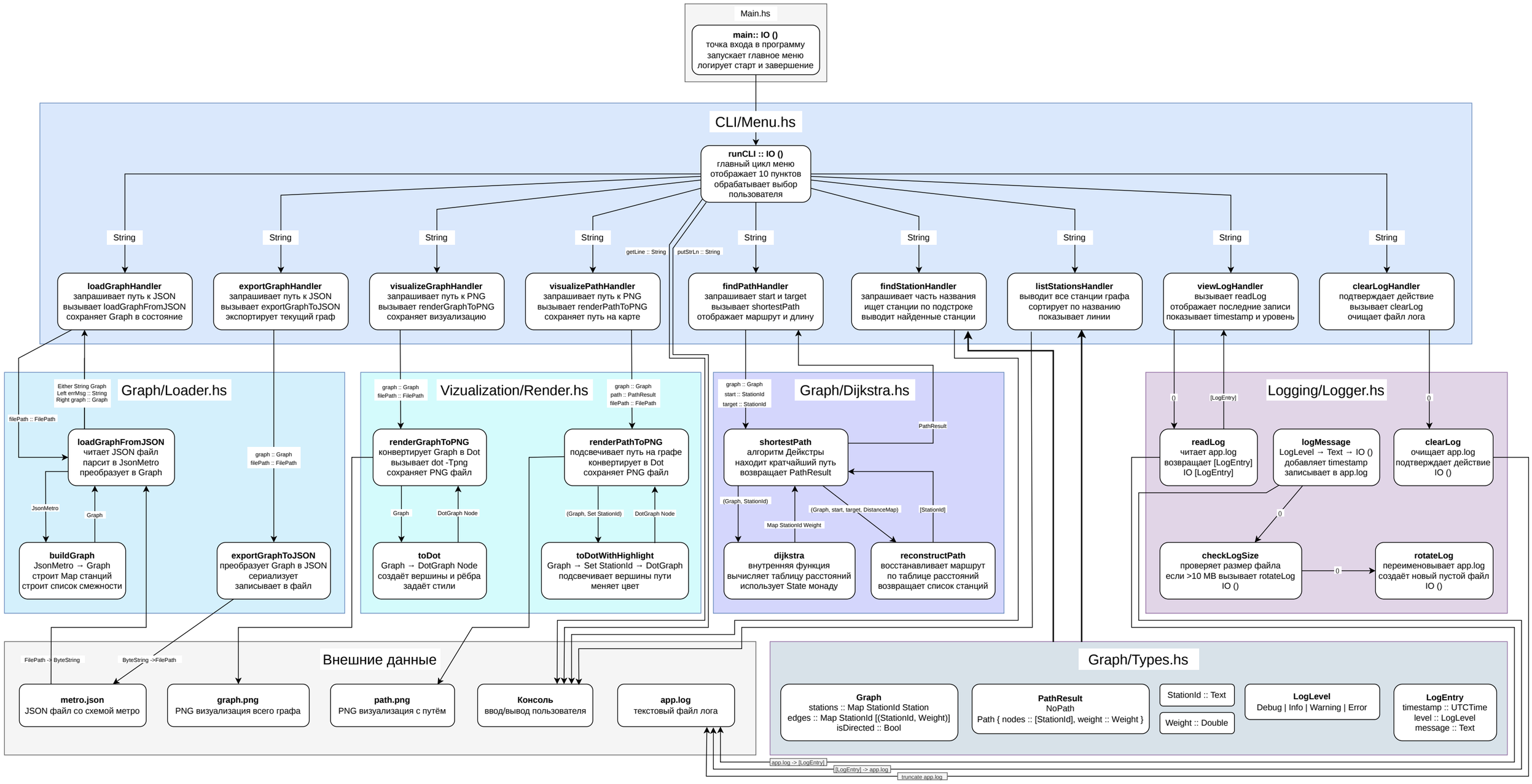


Рис. 2: Диаграмма модулей

1.2 Описание технических особенностей реализации проекта

1.2.1 Типы данных

Тип `StationName` определён как синоним `Text`, что позволяет использовать строковые идентификаторы станций. Тип `Weight` является обёрткой над `Double` и представляет вес ребра — расстояние между станциями в километрах. Благодаря использованию `newtype` с автоматическим выводом экземпляра `Num`, веса можно складывать и сравнивать, как обычные числа, но при этом они остаются отдельным типом, что предотвращает случайное смешивание с другими числовыми значениями.

Тип `Graph` представляет взвешенный неориентированный граф. Он содержит словарь станций (`stations`), где ключом является идентификатор станции, а значением — запись `Station`, хранящая название, линию и географические координаты, и список смежности (`adjacency`), отображающий каждую станцию на список соседей с весами. Такое представление обеспечивает постоянное время доступа к информации о любой станции и эффективный обход рёбер при работе алгоритма.

1.2.2 Алгоритм Дейкстры в чисто функциональном стиле

Реализация алгоритма Дейкстры выполнена в чисто функциональном стиле с использованием неизменяемых структур данных `Map` и `Set` из библиотеки `containers`. Алгоритм поддерживает ориентированные и неориентированные графы, возвращая результат типа `PathResult`, который может быть либо `NoPath` (путь не найден), либо `Path` с перечнем станций и общей длиной маршрута.

Ключевая особенность реализации — использование `Map` в качестве приоритетной очереди. На каждом шаге из очереди извлекается вершина с минимальным расстоянием (операция `Map.deleteFindMin`), после чего происходит релаксация всех её соседей. Такой подход работает для графов транспортных сетей, число вершин в которых редко превышает несколько сотен, и отличается простотой реализации в чисто функциональном стиле.

1.2.3 Вычисление расстояний по координатам

Для вычисления расстояний между станциями используется формула гаверсинусов, реализованная в функции `haversineDistance`. Эта формула учитывает кривизну земной поверхности и позволяет с высокой точностью определить расстояние между двумя точками на сфере по их широте и долготе. Вычисленное значение округляется до одного знака после запятой.

1.2.4 Парсинг и генерация JSON с помощью библиотеки aeson

Для работы с форматом JSON используется библиотека `aeson`. Определены типы данных `JsonStation`, `JsonLine` и `JsonMetro`, точно отражающие структуру входных JSON-файлов. Экземпляры `FromJSON` и `ToJSON` реализованы как вручную, так и с использованием автоматического вывода, что позволило гибко обрабатывать поля, которые могут отсутствовать в файле (например, координаты или пересадки). Поле `order` используется для правильной последовательности станций на линии, а поле `transfers` — для задания пересадочных узлов.

Особое внимание уделено проверке ссылочной целостности: функция `validateGrap` проверяет, что все станции, указанные в пересадках, действительно существуют в загруженном графе. При обнаружении ошибки возвращается содержательное сообщение с указанием некорректных связей. Экспорт графа в JSON сохраняет полную информацию: станции, линии, координаты и пересадки, что обеспечивает возможность восстановления графа без потерь.

1.2.5 Логирование с ротацией файлов

Модуль `Logger` реализует логирование всех значимых действий пользователя с временными метками и уровнями логирования (`Info`, `Warning`, `Error`). При каждом вызове `logMsg` в файл `app.log` добавляется строка формата «временная метка [уровень] сообщение». Перед записью проверяется текущий размер лог-файла: если он превышает 10 МБ, выполняется ротация — старые записи сохраняются в файл `app.log.old`, а основной лог-файл очищается. Это гарантирует, что лог не разрастётся бесконтрольно и не займёт весь диск.

1.2.6 Визуализация через Graphviz

Для сохранения визуализации графа в формате PNG используется внешняя программа `dot` из пакета Graphviz. В модуле `Render` реализована генерация DOT-описания графа на основе внутреннего представления. При этом:

- Станции отображаются в виде прямоугольников (`shape=box`) с подписью-названием.
- Рёбра между соседними станциями подписываются расстоянием в километрах.
- Пересадки (рёбра с весом 0) выделяются зелёным цветом.
- При визуализации найденного маршрута соответствующие станции выделяются красным цветом и утолщённой рамкой.

Генерация DOT-файла выполняется Haskell, после чего вызывается внешний процесс `dot` для конвертации в PNG.

1.2.7 Консольный интерфейс и обработка ошибок

Консольный интерфейс реализован в виде циклического меню с 10 пунктами, соответствующими функциональным требованиям технического задания. Все действия пользователя логируются, а ввод проверяется на корректность: обрабатываются пустые строки, неверные номера пунктов меню, отсутствие загруженного графа перед поиском пути или визуализацией. При отсутствии пути между указанными станциями программа выводит соответствующее сообщение и не завершается аварийно.

1.2.8 Тестирование с использованием QuickCheck

Для проверки корректности алгоритмов используются тесты на основе библиотеки `QuickCheck`. В частности, тестируются следующие свойства:

- Путь из станции в саму себя всегда имеет нулевую длину и состоит из одной станции.
- Веса всех рёбер графа неотрицательны.
- Неориентированный граф симметричен (если есть ребро A—B, то есть и ребро B—A).
- Длина кратчайшего пути между двумя станциями неотрицательна.

Генераторы случайных графов (`Arbitrary Graph`) создают небольшие линейные графы без петель и с симметричными рёбрами, что обеспечивает надёжную проверку свойств. Все тесты успешно проходят, что подтверждает корректность реализации алгоритмов.

2 Примеры работы программы

В данном разделе приведены примеры работы программы, демонстрирующие выполнение всех функциональных требований технического задания.

2.1 Главное меню

При запуске программы пользователю отображается главное меню (рис. 2), содержащее 10 пунктов: загрузка графа, экспорт, поиск пути, список станций, поиск станции, визуализация графа, визуализация пути, просмотр лога, очистка лога и выход.

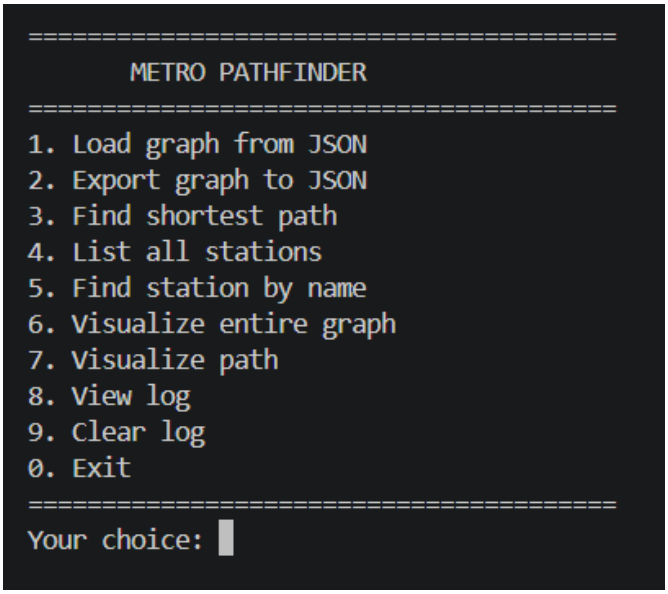


Рис. 2: Главное меню

2.2 Загрузка графа из JSON-файла

На рисунке 3 показан процесс загрузки графа метрополитена Санкт-Петербурга из файла `spb.json`. Программа успешно парсит JSON, извлекает станции с их координатами и пересадками. После загрузки выводится количество импортированных станций.

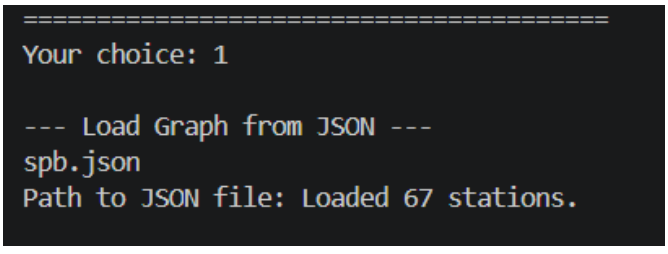


Рис. 3: Загрузка графа из JSON-файла

2.3 Просмотр списка станций

После загрузки графа пользователь может просмотреть список всех доступных станций (рис. 4). Список содержит названия станций в том порядке, в котором они были загружены.

```
========================================
Your choice: 4

--- List All Stations ---
Total stations: 67
  - Admiralteyskaya
  - Akademicheskaya
  - Avtovo
  - Baltiyskaya
  - Bukharestskaya
  - Chernyshevskaya
  - Chkalovskaya
  - Chyornaya Rechka
  - Devyatkino
  - Dostoevskaya
  - Elektrosila
  - Frunzenskaya
  - Gorkovskaya
  - Gostiny Dvor
  - Grazhdansky Prospekt
  - Kirovsky Zavod
  - Komendantsky Prospekt
  - Krestovsky Ostrov
  - Kupchino
  - Ladozhskaya
  - Leninsky Prospekt
  - Lesnaya
  - Ligovsky Prospekt
  - Lomonosovskaya
  - Mayakovskaya
  - Mezhdunarodnaya
  - Moskovskaya
  - Moskovskie Vorota
  - Narvskaya
  - Nevsky Prospekt
  - Novocherkasskaya
  - Obukhovo
  - Obvodny Kanal
  - Ozerki
  - Park Pobedy
  - Parnas
  - Petrogradskaya
```

Рис. 4: Просмотр списка станций

2.4 Поиск станции по части названия

Функция поиска позволяет найти станцию по фрагменту названия (рис. 5). При вводе части названия программа выводит все станции, содержащие указанную подстроку.

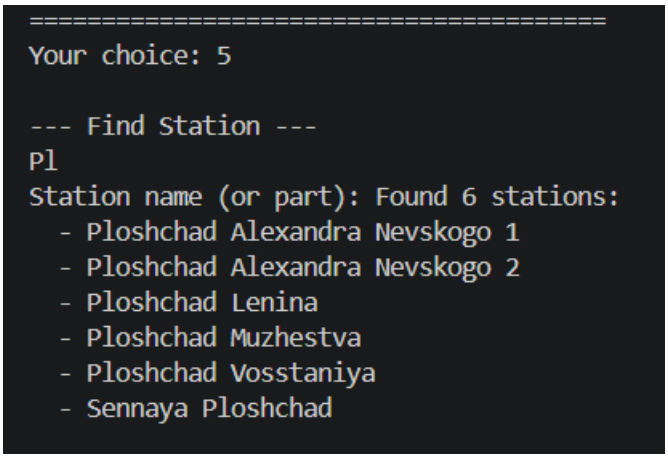


Рис. 5: Поиск станции по части названия

2.5 Поиск пути с пересадкой

На рисунке 6 продемонстрирован поиск пути между станциями «Pushkinskaya» и «Zvenigorodskaya» в Санкт-Петербурге, которые соединены пересадкой. Расстояние составляет 0.0 км, что соответствует переходу между линиями.

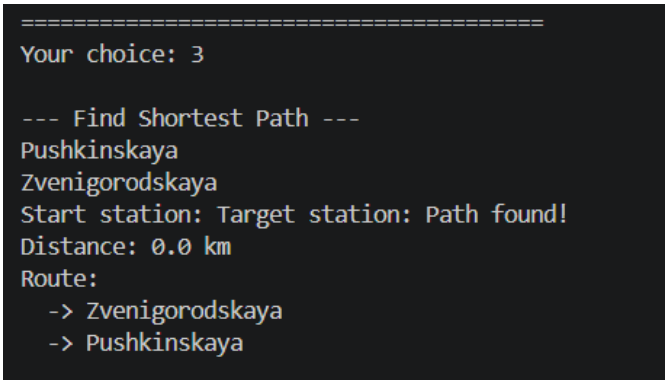


Рис. 6: Поиск пути с пересадкой

2.6 Поиск кратчайшего пути

На рисунке 7 показан результат работы алгоритма Дейкстры при поиске маршрута от станции «Parnas» до «Politekhnicheskaya» в Санкт-Петербурге. Программа выводит общую длину маршрута (31.5 км) и последовательность станций.

```
========================================
Your choice: 3

--- Find Shortest Path ---
Parnas
Politekhnicheskaya
Start station: Target station: Path found!
Distance: 31.5 km
Route:
  -> Politekhnicheskaya
  -> Ploshchad Muzhestva
  -> Lesnaya
  -> Vyborgskaya
  -> Ploshchad Lenina
  -> Chernyshevskaya
  -> Ploshchad Vosstaniya
  -> Vladimirskaya
  -> Dostoevskaya
  -> Spasskaya
  -> Sadovaya
  -> Sennaya Ploshchad
  -> Nevsky Prospekt
  -> Gorkovskaya
  -> Petrogradskaya
  -> Chyornaya Rechka
  -> Pionerskaya
  -> Udelnaya
  -> Ozerki
  -> Prospekt Prosveshcheniya
  -> Parnas
```

Рис. 7: Поиск кратчайшего пути

2.7 Визуализация графа

Программа сохраняет визуализацию графа в формате PNG (рис. 8). Станции изображаются в виде прямоугольников, рёбра подписаны расстояниями, пересадки выделены зелёным цветом.

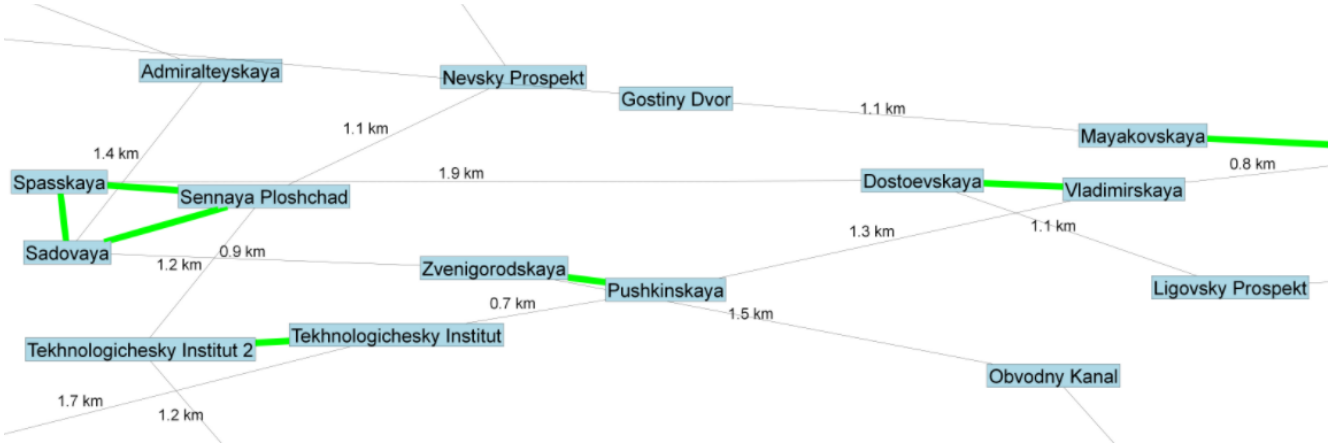


Рис. 8: Визуализация графа

2.8 Визуализация пути

При визуализации найденного маршрута (рис. 9) станции, входящие в путь, выделяются красным цветом и утолщённой рамкой, что позволяет наглядно увидеть маршрут на схеме.

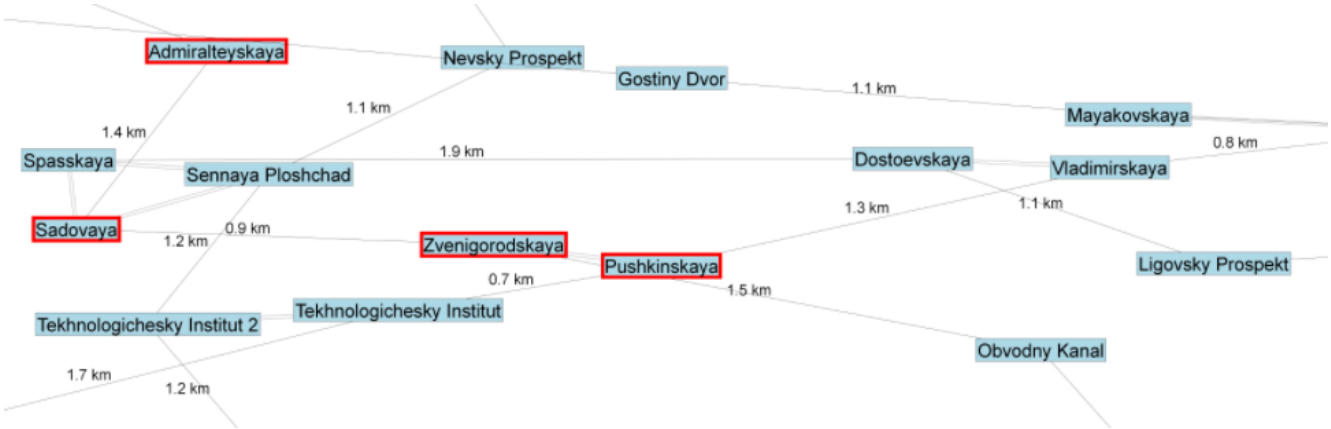


Рис. 9: Визуализация пути

2.9 Просмотр и очистка лога

Все действия пользователя фиксируются в файле `app.log` с временными метками (рис. 10). Пользователь может просмотреть последние записи лога и очистить его при необходимости.

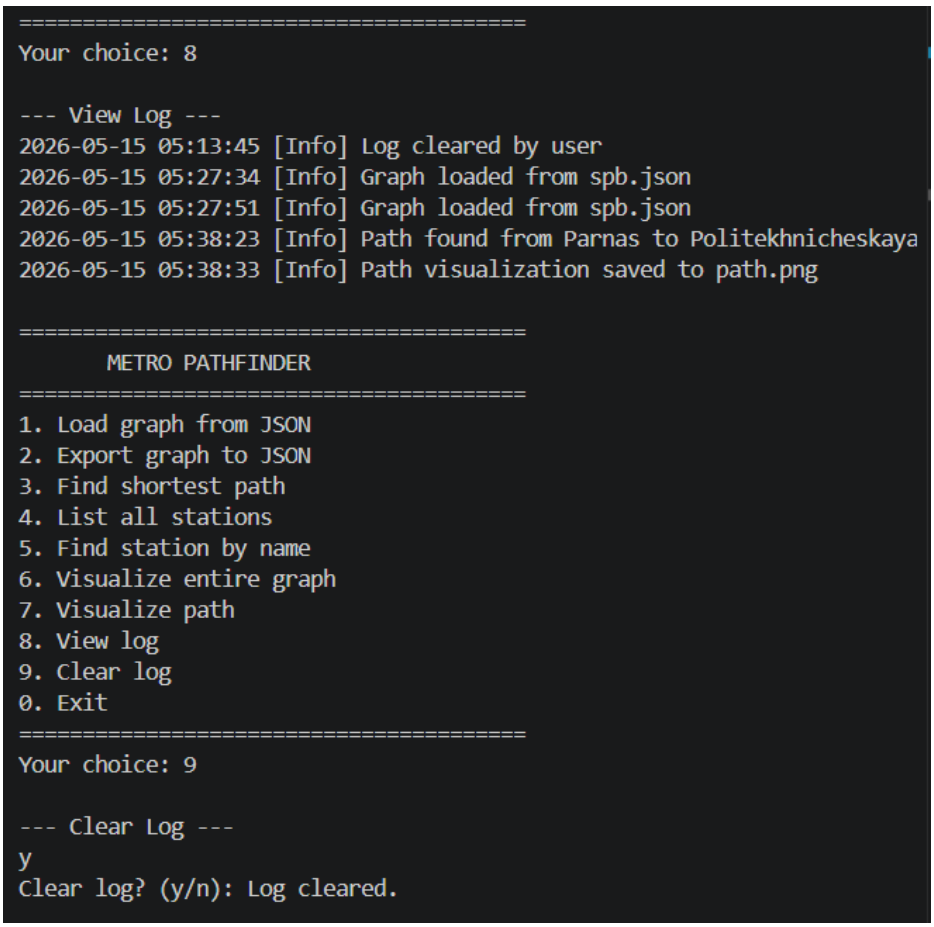


Рис. 10: Просмотр и очистка лога

2.10 Экспорт графа в JSON

После загрузки и возможной модификации граф может быть экспортирован в JSON-файл (рис. 11). Экспортированный файл сохраняет полную информацию о станциях, линиях, координатах и пересадках.

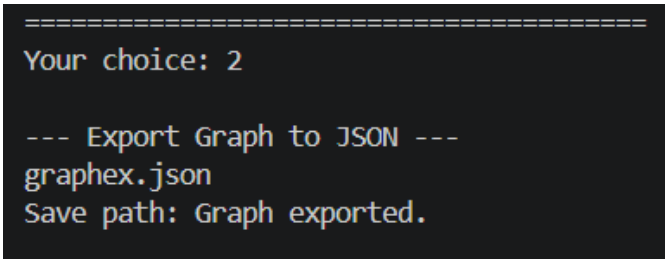


Рис. 11: Экспорт графа в JSON

3 Руководство по эксплуатации для пользователя

Для работы программы необходимы следующие компоненты:

- **Система сборки Stack** — для автоматической загрузки зависимостей и компиляции проекта.
- **GHC (Glasgow Haskell Compiler)** — версия 9.6.6 или выше.
- **Graphviz** — программа `neato` для генерации PNG-изображений графов (скачивается отдельно с `https://graphviz.org/download/`).

После клонирования репозитория команда `stack build` автоматически загружает необходимую версию GHC и все зависимости (aeson, containers, text, time, directory, bytestring, filepath, process). Graphviz необходимо установить вручную и добавить в системную переменную PATH.

3.1 Входные данные

Программа работает с JSON-файлами транспортных сетей. Формат файла должен соответствовать следующей структуре:

```
{
 "name": "Название города",
 "lines": [
 {
  "name": "Название линии",
  "hex_color": "CD0505",
  "stations": [
  {
   "name": "Название станции",
   "order": 0,
   "lat": 53.20969,
   "lng": 50.13442,
   "transfers": ["Станция пересадки"]
  }
  ]
 }
 ]
}
```

Обязательные поля: `name` (станции), `order` (порядковый номер на линии). Поле `transfers` используется для указания пересадочных станций (вес ребра будет равен 0). Поля `lat` и `lng` используются для вычисления реальных расстояний между станциями по формуле гаверсинусов.

Примеры готовых JSON-файлов: `samara.json`, `kiev.json`, `spb.json`.

3.2 Запуск программы

Программа запускается из корневой директории проекта командой:

```
stack run
```

При запуске отображается главное меню (Рис. 1):

```
========================================
METRO PATHFINDER
========================================
1. Load graph from JSON
2. Export graph to JSON
3. Find shortest path
4. List all stations
5. Find station by name
6. Visualize entire graph
7. Visualize path
8. View log
9. Clear log
0. Exit
========================================
Your choice:
```

3.3 Загрузка графа из JSON-файла

Для загрузки графа необходимо выбрать пункт меню `1`. Программа запросит путь к JSON-файлу.

После успешной загрузки граф становится доступен для всех последующих операций. При ошибке загрузки (неверный формат, отсутствие файла, некорректные пересадки) программа выведет сообщение с описанием проблемы.

3.4 Экспорт графа в JSON-файл

Выбрав пункт меню 2, пользователь может сохранить текущий граф в JSON-файл. Программа запросит путь для сохранения.

Экспортированный файл содержит полную информацию о станциях, линиях, координатах и пересадках.

3.5 Просмотр списка станций

Пункт меню 4 отображает список всех станций загруженного графа.

3.6 Поиск станции по части названия

Пункт меню 5 позволяет найти станцию по фрагменту названия.

3.7 Поиск кратчайшего пути

Для поиска маршрута между двумя станциями используется пункт меню 3. Программа последовательно запрашивает начальную и конечную станции.

При поиске пути между станциями, соединёнными пересадкой, расстояние будет равно 0.0 км.

Если путь между указанными станциями отсутствует, программа выводит сообщение: "No path found."

3.8 Визуализация графа

Пункт меню 6 сохраняет визуализацию всего графа в формате PNG. Для этого необходимо ввести название файла с расширением .png.

Станции отображаются в виде прямоугольников, рёбра подписаны расстояниями в километрах.

3.9 Визуализация пути

Пункт меню 7 сохраняет визуализацию графа с выделением найденного маршрута. Для этого необходимо ввести название файла с расширением .png.

Станции, входящие в маршрут, выделяются красным цветом и утолщённой рамкой. Для работы этой функции необходимо предварительно найти путь (пункт меню 3).

3.10 Просмотр лога

Пункт меню `8` отображает последние записи лог-файла.

Лог-файл `app.log` хранится в корневой директории проекта. Максимальный размер лога — 10 МБ, при превышении выполняется ротация (старые записи сохраняются в `app.log.old`).

3.11 Очистка лога

Пункт меню `9` позволяет очистить лог-файл.

3.12 Выход из программы

Пункт меню `0` завершает работу программы.

3.13 Обработка ошибок

Программа корректно обрабатывает все ошибочные ситуации и не завершается аварийно. При загрузке несуществующего файла, вводе пустого пути или неверного названия станции выводится соответствующее сообщение об ошибке. При отсутствии загруженного графа или ненайденном маршруте программа сообщает об этом и возвращает пользователя в главное меню. Все ошибки фиксируются в лог-файле с указанием уровня (Error или Warning).

Заключение

В ходе выполнения курсовой работы разработан программный комплекс на языке Haskell для поиска и визуализации кратчайших маршрутов в транспортных сетях. Реализованы все модули согласно ТЗ: типы данных для взвешенных графов, импорт/экспорт JSON (aeson), алгоритм Дейкстры с вычислением расстояний по формуле гаверсинусов, логирование с ротацией лога (10 МБ), визуализация графа и маршрута в PNG через Graphviz, консольное меню с десятью пунктами и тесты QuickCheck.

Программа позволяет загружать схемы метро из JSON, экспортировать граф, находить кратчайшие пути с учётом пересадок, просматривать и искать станции, сохранять визуализации, а также вести лог действий. Все функциональные требования выполнены. Программа протестирована на данных метрополитенов Самары, Киева и Санкт-Петербурга, работает стабильно и корректно обрабатывает ошибки ввода.

История коммитов:

```
Commits on March 13, 2026
4c76c65 - 1 версия тз

Commits on March 19, 2026
b5515a4 - 2 версия тз

Commits on April 3, 2026
0ee4ded - 3 версия тз

Commits on April 4, 2026
d4534ae - 4 версия отчета (итоговое тз для печати)

Commits on April 9, 2026
3d754b5 - 5 версия отчета

Commits on April 16, 2026
57a63b4 - Версия для печати

Commits on April 23, 2026
3a50c44 - Добавлена диаграмма модулей
1bc2cac - Исправлено введение

Commits on May 10, 2026
86eef8d - создан проект и добавлены типы данных
29a86f0 - добавлен модуль логирования
```

```
Commits on May 12, 2026
08695a8 - добавлены типы JsonStation, JsonLine, JsonMetro
673cc86 - добавлены функции loadGraphFromJSON, buildGraph

Commits on May 13, 2026
3e8c42e - добавлено консольное меню

Commits on May 14, 2026
b392ca4 - добавлена функция загрузки из json
340a7f4 - добавлены координаты станций
552bbc2 - добавлен экспорт графа в JSON
f74a0d7 - добавлена визуализация PNG через graphite

Commits on May 15, 2026
9a8dbf7 - интерфейс исправлен на более удобный
```

Диаграмма Ганта

Сравнение плановых и фактических сроков выполнения этапов представлено на рисунке 12.

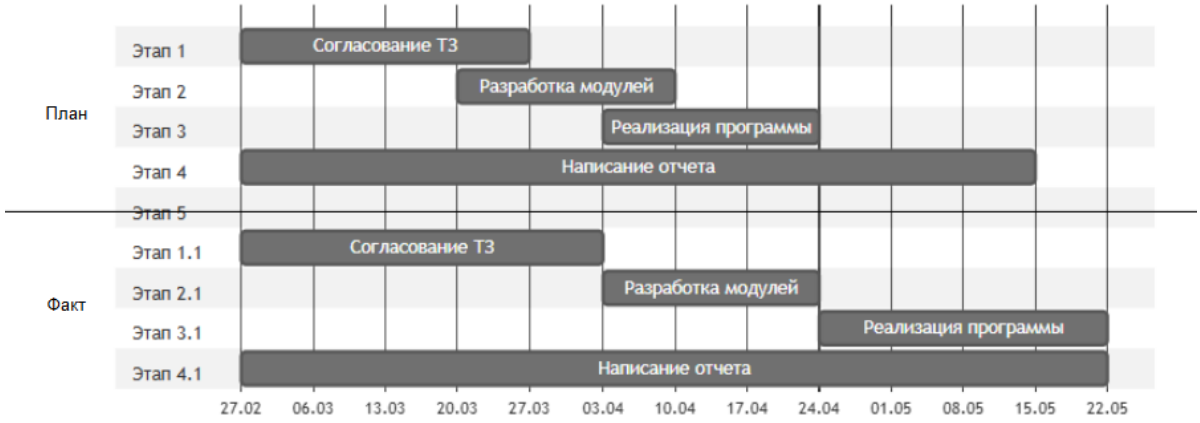


Рис. 12: Сравнение плановых и фактических сроков выполнения этапов

Список литературы

[1] Фролов Ю.С., Голицынский Д.М., Ледяев А.П. Метрополитены : учебник / под ред. Ю.С. Фролова. – М. : Желдориздат, 2001. – 528 с.

[2] Пашков Н.Н. Логистические транспортные системы : учебное пособие. – М. ; Вологда : Инфра-Инженерия, 2024. – 260 с.

[3] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. – М. : Вильямс, 2013. – 1328 с.

[4] Банина Н.В. Экономико-математическое моделирование транспортных процессов : лабораторный практикум. – Иркутск : ИрГУПС, 2017. – 76 с.

[5] Булычева Н.В., Капский Д.В., Лосин Л.А. Математическое моделирование для оценки роли метрополитена в транспортных системах крупных городов // Системный анализ и прикладная информатика. – 2022. – № 3. – С. 29–42.

[6] Сомов Э.В., Тикунов В.С. Геоинформационное моделирование интегральной транспортной доступности при планировании приоритетных направлений развития транспортной инфраструктуры на примере метрополитена г. Москвы // Геодезия и картография. – 2014. – № 8. – С. 49–53.